

PARTIAL

RAND

1

HASH

PRIMALTY

AA

FPSPACE

NP



1 COMPUTATIONAL COMPLEXITY CLASSES

Ham de veure una gran diferenciació, i és que, existeixen 2 tipus de problemes:

- Decisionals: es compleix una certa condició $P(x)$ per una entrada $x \in I$? Sí o no
- Funcionals: es calcula un graf/camí/distància/etc... a partir d'una entrada $x \in I$. Calculen alguna cosa!

Aquests problemes poden ser decidibles o indecidibles.

[Classe de Problemes Decisionals]

$$P = O(n^k)$$

$$EXP = O(2^{n^k})$$

$$2EXP = O(2^{2^{n^k}})$$

$$L = LOGSPACE$$

$$PSPACE = O(n^k)$$

$$EXPSPACE = O(2^{n^k})$$

TEMPS

ESPAI

(també estan les variants com NP,)
NEXP, NL, NPSpace, ...

[Classe de Problemes Funcionals]

FP

FEXP

F2EXP

FL

FSPACE

FEXPSPACE

(també estan FNP, FNEP, ...)

Propietats

$$L \subseteq PSPACE \subseteq EXPSPACE$$

$$L \subseteq P \subseteq EXP$$

$$PSPACE \subseteq EXP$$

$$L \subseteq NL \subseteq P$$

$$P \subseteq NP \subseteq PSPACE = NPSPACE \subseteq EXP$$

NP i Lògica

NP és la classe de llenguatges verificables en temps polinòmic

Formalment:

$$L \in NP \iff L = \{x \mid \exists y R(x, y)\} \text{ on } y \equiv \text{testimoni}$$

$R \equiv \text{verificador}$
 $x \equiv \text{input}$

Problemes NP:

- **Compartor**: donat un enter n , és compost?
- **Subset-sum**: donat un set S i enter n , hi ha algun $X \subseteq S$ tal que la seva suma sigui n ?
- **Cicle Hamiltonià**: donat un graf G , és G Hamiltonià?
- **Factorització**: donat enter n , hi ha primers $p_1, \dots, p_k$ tal que $p_1 \times \dots \times p_k = n$?
 - $p_1, \dots, p_k$ és un certificat/testimoni
 - Verificador veu si n i mira si $n = p_1 \times \dots \times p_k$, si passava més que diria primers

Complements de llenguatges

Si C és una classe de problemes de decisió, $\text{Co-}C$ està format pels complements dels llenguatges de C . Per exemple: $L \in \text{Co-}C \iff \bar{L} \in C$

- Per TMs deterministes els llenguatges són tancats per complementari:

$$P = \text{Co-}P$$

$$\text{EXP} = \text{Co-EXP}$$

$$\text{PSPACE} = \text{Co-PSPACE}$$

- Per TMs NO deterministes no està tan clar:

$$\text{Co-NP} = \{x \mid \forall y R(x, y)\}$$

(s'ha de comprovar per tota l'eixida!)

Problema Co-NP :

- Primalitat:

$$\text{PRIME} = \{n \mid \forall a \leq n \quad n \bmod a \neq 0\} \quad (\text{Co-NP})$$

PRIME és el complement de $\text{COMPOSITE} \Rightarrow \text{PRIME} \in \text{Co-NP}$

PRIME també pertany a NP, llavors pertany als dos!

2 REDUCTIONS I

COMPLETENESS

Problema A é polinomialmente redutível "many-one" a B

|||

(fácil) $A \leq_m^{\uparrow} B$ (difícil)

[Si $A \leq_m^{\uparrow} B$ llavors: $x \in A \iff f(x) \in B$]

On f é uma função que permet passar d'una instància de A a una de B en temps polinòmic.

Propietats:

$A \leq B$; $B \in C \Rightarrow A \in C$

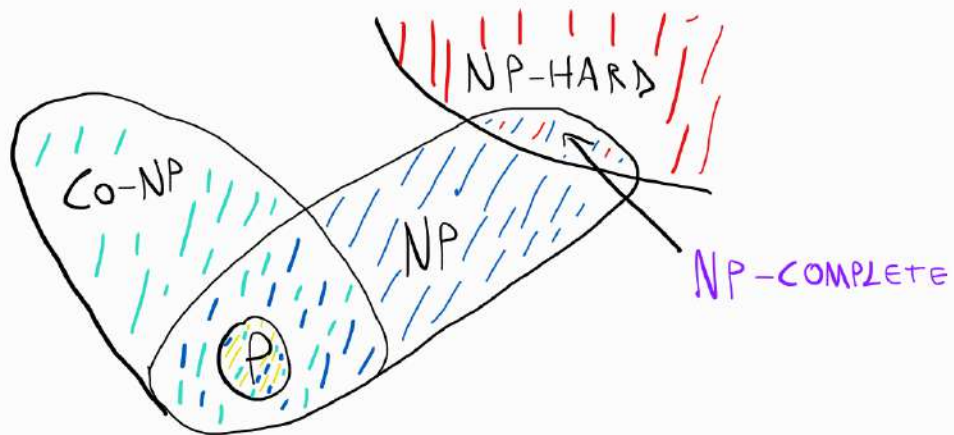
$A \leq B$ i $A \in C \Rightarrow B \in C$

[Si tenim A i $B \in \text{NP-COMPLET}$, si volem dir que $A \in \text{NP-COMPLET}$
llavors hem de fer: $B \leq A$ (B é més fàcil que A)]

- NP-HARD: Si $A \in \text{NP-HARD}$, llavors tot problema de NP es pot reduir a A. ($\forall B \quad B \leq A$)

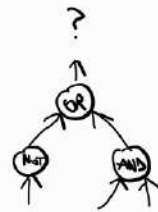
- NP-COMPLETE: $A \in \text{NP-HARD}$ i també $A \in \text{NP}$.

- El que es sap:



- D'acord, si $A \in \text{NP-COMPLETE}$ i $A \in P \iff P = NP$ 

- CIRCUIT-SAT $\in \text{NP-COMPLETE}$
(Cook-Levin's theorem)



SAT } Treballen sobre CNF! $\equiv \phi = (x_1 \vee x_2) \wedge (\dots) \wedge (\dots) \wedge (\dots)$
 k-SAT } Són NP-COMPLETE (amb $k \geq 3$)

DNF-SAT } Treballen amb DNF!
 k-DNF-SAT }

Problemas de Graphs

• CLIQUE:

- Entrada: $G = (V, E)$ e enter k
- Pergunta: existe um subgraph G' complet de k vertices?

• Independent Set (IS):

- Entrada: $G = (V, E)$ e enter k
- Pergunta: existe um $S \subseteq G$ com $|S| \geq k$ tal que que S n'atingui arestas? (verificar soln)

• Vertex Cover (VC):

- Entrada: $G = (V, E)$ e enter k
- Pergunta: existe um $S \subseteq G$ com $|S| \geq k$ tal que cada aresta $\in E$ te almeno um dos seus vertices em S ?
(cada aresta a de tenir almeno um extrem cobert)

- Si problema funcional $X_F \in P$ i problema decisonal $X_D \in NP$,
podem fer una reducció $X_D \leq X_F$.

Utilitzem algorisme de X_F , el A_F en l'algorisme A_D
per a una entrada x .

- Si $X_F \in NP$ i $X_D \in P$

Aquí utilitzem l'autoredutilibilitat!

Avem preguntant si o no a A_D amb l'entrada

Fent cerces dicotòmics o parant o true i false

3] PROBABILISTIC ALGORITHMS

- Generar pseudo-aleatori:

$$x_{n+1} = (a x_0 + b) \bmod m$$

↑ ↑ ↑
large seed low
↖ large integer

- Anàlisis probabilístic d'un algorisme determinista:

- 1 - Fixar una distribució en el set de inputs. La distribució serà uniforme, però no sempre. (inputs random equitativament)
- 2 - Llavors, el nombre de passos és una variable $T(n)$. Calcular el valor esperat com $\mu = E[T(n)]$. (Calcular esper a base prob.)
- 3 - Hem de demostrar la probabilitat de que amb probabilitat alta $T(n)$ està aprop de μ .

- Al dissenyar algorismes aleatoris, ara el la complexitat i els outputs són variables aleatòries.
- Als algorismes aleatoris es fan decisions random, i es segueix amb l'execució.

Tipus d'algorismes aleatoris

- Monte-Carlo: sempre para en temps finit, però pot donar resposta incorrecte. Si l'output és del tipus Si/no l'error pot estar en una banda o l'altra. Cal indicar probabilitat d'error.
- Las Vegas: resposta sempre correcta però pot no acabar/estar creatat.

És fàcil passar de Las Vegas a Monte-Carlo, però el revés no sempre és veritat.

- Permutacions de n elements $\Rightarrow n!$
- n elements amb x probabilitats $\Rightarrow x^n$ (x és 2^n)
- Permutació uniformement at random (U.A.R.) és generar totes les possibles permutacions amb la mateixa probabilitat: $\frac{1}{n!}$

Fisher-Yates Algorithm (Knuth's algorithm)

- S'utilitza per crear permutacions U.A.R. sobre un vector
- Primer passa el element segons posicions reals ($0 \dots n-1$)
- Després volent swap per generar U.A.R. permutació

Random-Perm(n)

for $i=0$ to $n-1$ do

- $\pi[i] = i$

end for

for $i=n-1$ to 1

- $j = \text{rand}(i+1)$ // entre $[0, i+1)$

- swap $\pi[j]$ i $\pi[i]$

end for

Que un element oculhi sent l'últim
é igual per a totz, de n fins a 2 :

$$\frac{1}{n} \cdot \frac{1}{n-1} \cdot \dots \cdot \frac{1}{2} = \frac{1}{n!}$$

$O(n)$

Checking Matrix Multiplication $\Theta(n^2)$ i no $O(n^3)$ com normalment

• Donades A, B, C matrius de $n \times n$, volem comprovar si $A \times B = C$

Freivald(A, B, C)

Esull u.a.n $r \in \{0, 1\}^n$ // coders de n bits random

Si $A \cdot (Br) = Cr$ llavors True

Sinó False

Pouem a vectors per
é més fàcil!

Si $AB=C$ sempre deu ser

Sinó pot fallar amb
prob $\leq \frac{1}{2} \cdot \pm$ de $\frac{1}{2^k}$ si
repetim k vegades!

4] MORE EXAMPLES

PROBABILITATS

• Minimum Cut Problem

- Donat un graf no dirigit $G=(V,E)$ un cut és una partició de V en S i $\bar{S}$.
- La capacitat d'un cut és el nombre d'arestes que tenen un extrem a S i l'altre a $\bar{S}$.
- Min cut és el cut amb mínima capacitat.

(en resum, el mínim nombre d'arestes que cal eliminar per separar el graf)

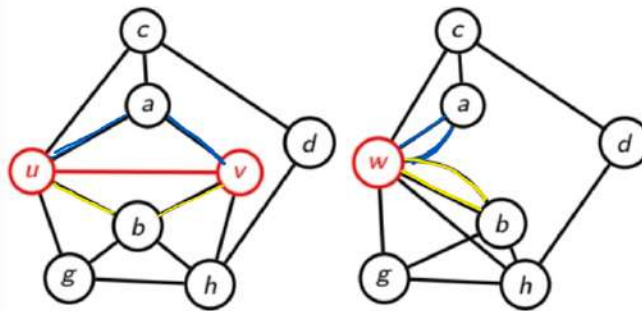
- De forma normal trobar Min cut triga $\left\{ \begin{array}{l} O(n^2 m) \text{ Ford Fulkerson} \\ O(nm) \text{ J. Edmonds} \\ O(nm + n^2 \log n) \text{ Stoer-Wagner} \\ \text{(no-flow amb precs)} \end{array} \right.$

Dirany Probabilistic de Karger

L'algorisme contrauria arestes de $G=(V,E)$, aquesta operació produirà un graf amb múltiples arestes però sense self-loops ($\emptyset$)

Contract ($e=(u,v)$)

- reemplaça u i v per super-node w
- preserva arestes, actualitza arestes veïnes per a que apuntin a w
- manté arestes paral·leles, però elimina self-loops.



Karger ($G=(V,E)$)

mentre $|V| > 2$ fa

esull n.a.n $e_i = (u,v) \in E$

$G = \text{contract}(e_i)$

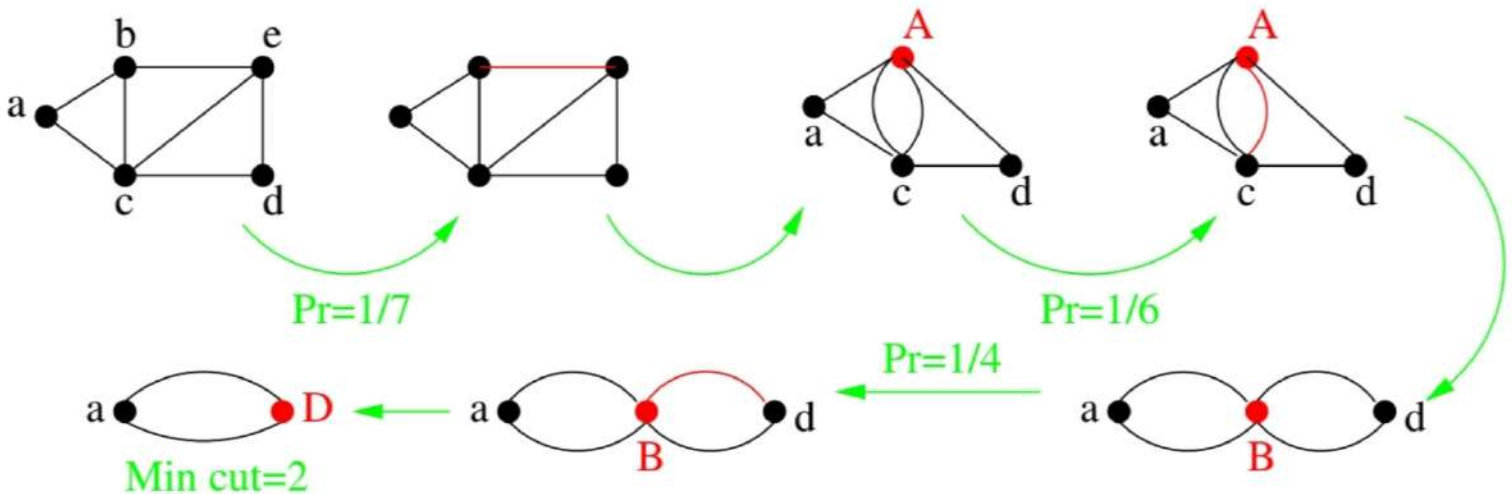
return edges entre els dos nodes

$\Theta(n^2)$

Karger's retorna sempre un cut

No garanteix el mínim

Mínim amb prob $\geq \frac{2}{n^2}$



• [MAX 3-SAT]

- 3-SAT fórmula é aquella fórmula Booleana CNF tal que cada clàusula té 3 literals i cada literal representa una variable diferent

$$(x_1 \vee \bar{x}_3 \vee x_4) \checkmark \quad (x_1 \vee \bar{x}_1 \vee x_2) X \quad (x_1 \vee x_2 \vee x_3 \vee x_4) X$$

- MAX 3-SAT donada una 3-SAT, trobar una assignació de veritat on el màxim de clàusules satisfetes.

↳ $\in$ NP-Hard. Farem algorisme aleatori que faci una bona assignació, encara que no sigui òptima.

FlipCoin SAT(X)

Per cada variable gira una moneda, assigna True si é cara, False d'altra banda

Probabilitat $x_i = 1 \in \frac{1}{2}$ i é independent de les altres.

Prob satisfen clàusula $P_n[z_i = 1] = 1 - \left(\frac{1}{2}\right)^3 = \frac{7}{8}$

Ullavors $E[z_i] = \frac{7}{8}$ i $E[z] = \sum_{i=1}^m z_i = \underline{\underline{\frac{7}{8} m}}$

Almenys farà centes $\frac{7}{8}$ clàusules!!!

5 PRIMALTY

Teoremes

• Gauss 1973: $\text{primers} \leq n \equiv \pi(n) \left[\pi(n) \approx \frac{n}{\ln n} \right]$

• Corollary: p_n és el n -th primer, $p_n \approx n \ln n$

• Teorema petit de Fermat:

$\hookrightarrow$ Per tot n primer i $\forall a \in \mathbb{Z}_n^+$, $\left[a^{n-1} \equiv 1 \pmod{n} \right]$

• Teorema de Lucas:

$\hookrightarrow$ Siguiu $a, n \in \mathbb{Z}^+$

$$\left[n \text{ primer} \iff a^{n-1} \equiv 1 \pmod{n} \wedge \forall q \text{ divisor primer de } (n-1), a^{\frac{n-1}{q}} \not\equiv 1 \pmod{n} \right]$$

Primality Problems

• Primality Problem: donat $n \in \mathbb{Z}^+$ amb N -bits, és n primer? Aquest problema pertany a P!

• Donat un enter n , podem generar en temps polinòmic el n -th primer?

• Donat un enter N , podem generar en temps polinòmic un primer de N bits?

Primality Problem

Algorisme naïf

for $a = 2, 3, \dots, \sqrt{n}$ do
 if $a | n$ then composite
return prime

Temps $O(\sqrt{n})$, però per entendre
l'ordre com $n = 2^{2024}$ el input

é $N = \log n$ i.e. $n = 2^N$.

Hauriem de buscar amb $O(\sqrt{n})$ divisors
de n , amb cada un un cost de check $O(N^2)$.

$T(N) = O(2^{\frac{N}{2}} N^2)$, molt lent!
↙ ↘
més i fins més cada un costa

n é un 'pseudoprime en base a ' si $a^{n-1} \equiv 1 \pmod n$

PseudoPrime(n)

$a = \text{rand}(1, n-1)$

si $(a^{n-1} \equiv 1 \pmod n)$ é pseudoprime

else no prime

$O(N^3)$

Els nombres de Carmichael són
nombres no primers que compleixen
la condició de sobre! Són nombres
molt rars (255 amb valor $< 10^8$).

Els Carmichael fan que, si tinem que no é primer, pot fallar si é base 2.

Si assumim la no existència dels Carmichael:

1- Si n é primer, sempre dona la resposta correcta !!
Si n no é primer falla amb prob. $\leq \frac{1}{2}$

2- É un algorisme que falla d'un costat (one-side), llavors si repetim
 k vegades la prob. é $\leq \frac{1}{2^k}$

• Teorema (tenint en compte Carmichael)

↳ $p > 2$ és primer i $n = p^k$, per $k \geq 1$, l'equació $x^2 \equiv 1 \pmod{n}$ té exacte 2 solucions, $x = -1$ i $x = 1$.

• Zorathaly: si existeix una anul quadrat no-trivial de 1 mod n , llavors n és compost

($x \in \mathbb{Z}$ és una anul no-trivial de 1 mod n si satisfi $x^2 \equiv 1 \pmod{n}$ però $x \not\equiv \pm 1 \pmod{n}$). 6 anul no-trivial de 1 mod 35

G. Miller (1976), M. Rabin (1980) (Algorisme aleatori)

• Si $x^2 \equiv 1 \pmod{n}$ exactament té com a solucions $x = \pm 1$ llavors n és primer.

• Si hi ha una altra solució n no és primer.

• Per veure si n és primer

- Aleatoriament escullim un $a < n$, si $a^2 \equiv 1 \pmod{n}$ llavors a és una anul no-trivial de 1 mod n , i n no és primer. Llavors n és un testimoni de la composició de n .

- D'altra banda, no pot ser a ra primer

- Repetir per reduir el error

• Mitjançant (a, n) // per veure si n és compost gràcies a 'a'.

↳ check $a^{n-1} \not\equiv 1 \pmod{n}$, o si no

↳ check $\exists m \in \mathbb{Z}_n^+$ tal que: $x = a^m$ és una anul no-trivial de 1 mod n

Witren (a, n)

Calcula t tal que: $n-1 = 2^t \cdot m$

$$X_0 = a^m \bmod n$$

for $i=1$ to t do

$$X_i = X_{i-1}^2 \bmod n$$

Si $(X_i = 1 \wedge X_{i-1} \neq 1 \wedge X_{i-1} \neq n-1)$ return true { X_i and no-divisor de 1}

Si $X_t \neq 1$ return true {Fermat falla}

return false { a no es un testimoni}

Miller-Rabin (n, s)

for $i=1$ to s do

$$a = \text{random}(1, n-1)$$

Si (Witren(a, n) = true) no es primo {100%}

return es primo {con segun}

$$O(s N^3)$$

↳ Algoritme aleatori polinòmic per dir si un $n \in \mathbb{Z}$ és primer.

L'input és n i el nombre s de $a \in \mathbb{Z}$ que provar si és testimoni.

$$\text{Probabilitat de fallar} < \frac{1}{2^s}$$

Com generar un primer llong n

- 1) Escollir un nombre n imparell de N bits
- 2) Run Miller-Rabin en n si passa cent put n
- 3) Si no repetir el procés un mlt s vegades.

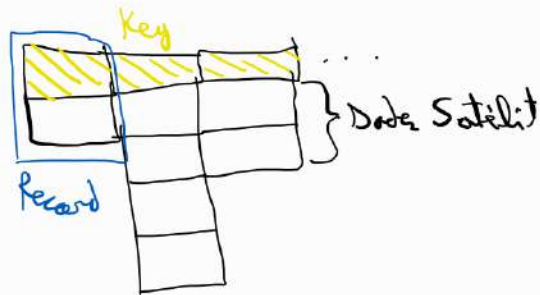
Prob $\frac{1}{N}$ n seria primer.

En pràctica, $s=50$
és suficient !!

6 HASHING

Recordatori Estructures de Dades

Un univers U (conjunt de dades) on cada dada té una Key.



Tipus de Estructures de Dades {

- Array / Vector
- Linked List
- Stack (LIFO): push and pop
- Queue (FIFO): enqueue and dequeue
- Deque: push, pop, enqueue, dequeue
- Priority Queue: insertions, deletions, find Max/Min
- Hash Tables

Estructures de Dades per a conjunts dinàmics

• DICTIONARY

↳ E.D per a mantenir $S \subset U$ amb les operacions:

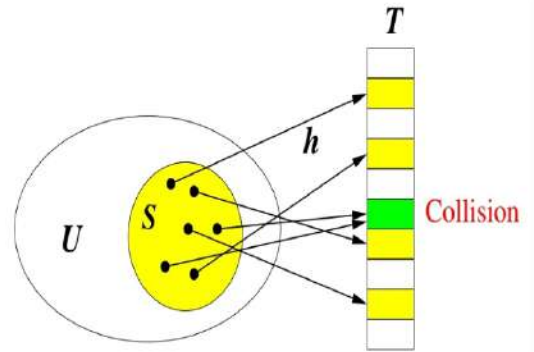
- Search(k): decidir si $k \in S$
- insert(k): $S := S \cup \{k\}$
- delete(k): $S := S \setminus \{k\}$

Hashing

És una E.D que suporta operacions de diccionari en un univers de keys numèriques.

Definim una hash table $T[0, \dots, m-1]$

→ amb funció de hash $h: U \rightarrow T[0, \dots, m-1]$



Funció de Hash Simple uniforme

- Valen guardar màxim n keys en una H.T de m slots.
- ⚠ • La performance de hashing depen de com de bé h distribueix les keys en els m slots.

- h é simple uniforme si assigna una key amb la mateixa probabilitat a qualsevol slot!

→ Obtenim un factor de càrrega $\alpha = \frac{n}{m}$, la mitjana de keys per slot. (si $n=m \Rightarrow \alpha=1$!)

- Com decidim h ? $h(k) \equiv$ an t'entera

→ Si keys en interval $[0, 1)$, llavors: $h(k) = \lfloor mk \rfloor$

→ Si keys en interval $[s, t)$ crealem amb $\frac{1}{t-s}$ i

$$\text{llavors: } h\left(\frac{k}{t-s}\right) = \left\lfloor m \cdot \frac{k}{t-s} \right\rfloor$$

• Mètode per divisió

→ Escull un primer el més allunyat possible d'una potència de 2. (p.e. 73 no és bo, o més de 16)

$$h(k) = k \bmod m$$

→ Ràpid de calcular $\Theta(1)$, e fer $k \% m$

→ Amb compte! Si $m = 2^n$ la funció de hash no depen de tots els bits de k !

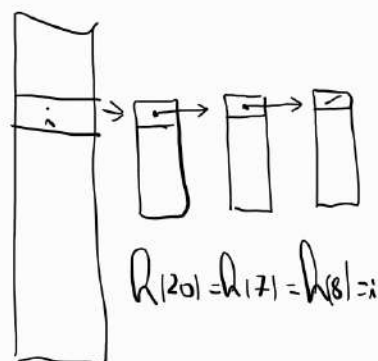
If $r = 6$ with $k = 1011000111\underbrace{011010}_{=h(k)}$ → Només depen 6 últims bits!

→ Amb n molt llargues, podem utilitzar: mod arithmetic + Horner's method

Resolució de col·lisions

• Per cada $T[i]$ fer una linked list, on posarem les dades que tenen $h(k) = i$

• Trobar element en $h(k) =$ llargada llista en $h(k)$



Cost operació de diccionari utilitzant hashing

- Insertion $\rightarrow \Theta(1)$
- Search $\rightarrow O(\text{length list})$
- Delete $\rightarrow O(\text{length list})$

Si h és simple uniforme, llavors tenim $O(\alpha) = O\left(\frac{n}{m}\right)$

Si $m = \Theta(n)$ llavors és $\Theta(1)$!

Universal Hashing ☺ (Per evitar eventes adverses de la hash table)

- Sigui $\mathcal{M}$ l'unió de keys, i sigui $\mathcal{H}$ la col·lecció de funcions de hash per a una hash table $T[0, \dots, m-1]$.

- $\mathcal{H}$ és universal si $\forall x, y \in \mathcal{M} : x \neq y$ llavors:

$$|\{h \in \mathcal{H} \mid h(x) = h(y)\}| \leq \frac{|\mathcal{H}|}{m}$$

(nombre funcions hash iguals $\leq \frac{|\mathcal{H}|}{m}$)

- Equivalentment, $\mathcal{H}$ és universal si $\forall x, y \in \mathcal{M} : x \neq y$ i tota h escollida uniformement de $\mathcal{H}$ tenim:

$$P_{\mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m}$$

• Teorema

↳ Escull u.a.n una h de $\mathcal{H}$ i construeix una taula $T[0, \dots, m-1]$ per a n keys. Per tota key x sigui C_x una variable random que conta les col·lisions que té x amb altres keys y en T . Llavors:

$$E[C_x] \leq \frac{n}{m}$$

Vist tot això, la funció de hash universal demostren la estratègia de l'observari!

Construcció d'una família universal: $\mathcal{H}$

Si sigui M l'univers de keys, i sigui N el màxim valor de les keys. Tenim hash table de m posicions $T[0, \dots, m-1]$.

- Escull un primer p , $N < p < 2N$

$$\text{Llavors } M \subset \mathbb{Z}_p = \{0, 1, \dots, p-1\}$$

- Defineix $\mathcal{H} = \{h_{a,b} \mid a, b \in \mathbb{Z}_p, a \neq 0\}$

- Per escollir u.a.n $h \in \mathcal{H}$ llavors escull u.a.n un $a \in \mathbb{Z}_p^+$ i un $b \in \mathbb{Z}_p^+$.

• Donada una key x , defineix $h_{a,b}(x)$:

$$h_{a,b}(x) = ((ax+b) \bmod p) \bmod m$$

Propietats de $\mathcal{H}$

① $h_{a,b}: \mathbb{Z}_p \rightarrow \mathbb{Z}_m$

② $|\mathcal{H}| = p(p-1)$ poder seleccionar a en $p-1$ possibilitats
i b en p

③ Especificar una $h \in \mathcal{H}$ requereix $O(\log p) = O(\log N)$ bits

④ Per seleccionar $h \in \mathcal{H}$ seleccionem a, b independentment
i M.a.n de $\mathbb{Z}_p^+$ i de $\mathbb{Z}_p$ respectivament.

⑤ Evaluar $h(x)$ é ràpid!

Bloom Filter

- Volem E.D on poguem fer insercions i preguntes per pertinença ràpidament. On utilitzem molt poc espai.
- Burton Bloom va inventar el [Bloom filter]
 - ↳ Una E.D tipus hash on cada registre de T és 1 bit!

• La seva definició:

- ↳ Tenim $S = \{x_1, \dots, x_n\}$ (la info)
- ↳ Creem una taula de hash $T[0, \dots, m-1]$ de registres de mida 1 bit. $\exists$ alguna funció hash $h: S \rightarrow [0, \dots, m-1]$
- ↳ Inicialment tots els bits a 0 en T .

Funcions de Bloom filter

Create table Insert (x)

$$T[h(x)] = 1$$

Notice: once we have hashed S into T , we **erase** S .

(primer fem hash tot a T ,
i esborrem S)



Query table inS(y)

if $T[h(y)] == 1$ **then**
 return Yes

else

return No

end if

(ara podem fer consultes)

• Característiques:

- Ocupa $O(m)$ en memòria
- Consulta en $O(1)$
- Es pot augmentar però no pots fer delete
- Està $y \in S$?

1- Bloom filter contesta bé si $y \in S$

2- Si $y \notin S$ pot donar fals positiu! $T[h(y)] = 1$ algun!

$$P_n[T(w)=0] = e^{-\frac{m}{n}}$$

$$P_n[T(w)=1] = 1 - e^{-\frac{m}{n}} \quad (\text{prob fals positiu!})$$

↳ Per baixar aquesta prob. cal fer $m > n$

• Amplificació:

- En comptes de tenir només una funció de hash h podem tenir k funcions de hash, $h_0 \dots h_{k-1}$.

D'aquesta forma baixem prob. fals positiu al mirar el resultat de totes les h_i . Si una dona 0 segueix que $y \notin S$. Si totes 1 llavors segurament $y \in S$.

- Així ho guardem a memòria $T[k, m]$, i anirem mirant de $T[0, h_0(y)] \dots T[k-1, h_{k-1}(y)]$.

$$\left[- \text{Ara la nova prob. fide partiu e: } \left(1 - e^{-\frac{km}{n}} \right)^k \right]$$

$$- \text{La kapt serà: } \underline{\underline{\ln(2) \cdot \frac{n}{m}}} \text{ (trabada derivant)}$$

$$- \text{Amb kapt prob fide partiu e: } 0.6185^{\frac{n}{m}}$$

- Si $m = cn$ llavors tenim prob. constant !!!

Factors de Probabilitat

↳ Derivats de Markov

• Lemma: Si $X \geq 0$ e una v.a., per una constant $a > 0$

$$P_n[X \geq a] \leq \frac{E[X]}{a}$$

• Corollari:

Si $X \geq 0$ e una v.a., per una constant $b > 0$

$$P_n[X \geq bE[X]] \leq \frac{1}{b}$$

↳ Desigualtat de Chebyshev

- Si pots calcular la variància d'una v.a llavors pots calcular σ i tenir millors bound de concentració.

• Teorema:

sigui X una v.a amb $E = \mu$ i $\sigma > 0$, llavors per tota $a > 0$:

$$P_n[|X - \mu| \geq a\sigma] \leq \frac{1}{a^2}$$

↳ Límits de Chernoff

- El límit de Chernoff es pot utilitzar quan la v.a X és la suma de independents Poisson, on cada X_i té prob. success de p_i . Quan totes les p_i són iguals estem en el cas de Bernoulli.

• Teorema:

sigui $\{X_i\}_{i=0}^n$ el conjunt independent de Poisson, amb $P_n[X_i = 1] = p_i$. Llavors, si $X = \sum_{i=0}^n X_i$ i $\mu = E[X]$ tenim:

$$\textcircled{1} P_n[X \leq (1-\delta)\mu] \leq \left(\frac{e^{-\delta}}{(1-\delta)^{(1-\delta)}} \right)^\mu$$

$\delta \in (0, 1)$

$$\textcircled{2} P_n[X \geq (1+\delta)\mu] \leq \left(\frac{e^{\delta}}{(1+\delta)^{(1+\delta)}} \right)^\mu$$

$\delta > 0$

• Corollary 1:

$$\textcircled{1} P_n [X \leq (1-\delta)\mu] \leq e^{\frac{-\mu\delta^2}{2}}, \delta \in (0,1)$$

$$\textcircled{2} P_n [X \geq (1+\delta)\mu] \leq e^{\frac{-\mu\delta^2}{3}}, \delta \in (0,1]$$

• Corollary 2:

$$P_n [|X - \mu| \geq \delta\mu] \leq 2e^{\frac{-\mu\delta^2}{3}}$$

El Problema de Comptar Elements Diferents

• Donat un stream S , retorna $|\{j \mid f_j > 0\}|$, on f_j és la freqüència de j en S .

• Per poder solucionar el problema en espai sublineal, necessitem algorismes / E.D probabilístiques.

• Aproximació (ϵ, δ)

→ Sigui $A(S)$ l'output de l'algorisme A en input S , $A \in \text{a.a.}$

→ Sigui $\phi(S)$ la funció que A hauria de computar. (ϕ_{ideal})

→ A és una (ϵ, δ) -aproximació a ϕ si:

$$P_n \left[\left| \frac{A(S)}{\phi(S)} - 1 \right| > \epsilon \right] \leq \delta$$

↳ A és una (ϵ, δ) -aproximació-additiva a ϕ si:

$$P_{\cap} [|A(s) - \phi(s)| > \epsilon] \leq \delta$$

- Quan $\delta = 0$, A ha de ser determinista
- Quan $\epsilon = 0$, A ha de ser un algorisme exacte.
- Algorisme:

↳ Per a un enter $p > 0$, sigui $\text{zeros}(p)$ el nombre de zeros al final de la representació binària de p .

$$\text{zeros}(p) = \max \{i \mid 2^i \text{ divideix } p\}$$

↳

```
1: Count-Dif(stream s)
2: Choose a random hash function  $h : [n] \rightarrow [n]$  from a universal family
3: int  $z = 0$ 
4: while not  $s.\text{end}()$  do
5:    $j = s.\text{read}()$ 
6:   if  $\text{zeros}(h(j)) > z$  then
7:      $z = \text{zeros}(h(j))$ 
8:   end if
9: end while
10: Return  $2^{z + \frac{1}{2}}$ 
```

Assumint hi ha d elements diferents, l'algorisme calcula $\max \text{zeros}(h(i))$ com a màxim aproximació de $\log d$.

L'algorisme dona una $(2, \frac{\sqrt{2}}{3})$ -aproximació

